

Controller控制器

- 4-1: 認識Controller

建立Controller	4-2
製作與使用Action	4-3
常見Content-Type (MIME Type) 對照表	4-4
Action的回傳種類-ContentResult	4-5
Action的回傳種類-ViewResult	4-6
Action的回傳種類-FileResult	4-7
Action的回傳種類-JsonResult	4-8
Action的回傳種類-RedirectResult	4-10
動作過濾與動作屬性 (Action / Attribute)	4-12

- 4-2: 與View的資料傳遞

資料傳遞至View的方式	4-13
ViewData	4-14
TempData	4-15
ViewBag	4-16
比較ViewData、TempData與ViewBag	4-18

- 4-3: Area: Area

Area是什麼	4-22
建立 Areas 結構	4-23
設定Area路由	4-26

Module 04

建立Controller

- **Controller** 負責 控制網站的整體流程，是 使用者介面（**View**）與邏輯層（**Model**）之間的協調者。
 - **Controller** 本身為類別（**Class**），存放於 **.cs** 檔案中。
 - **Controller** 內會定義多個公開方法（**public method**），稱為**Action**。
 - **Action** 方法 用來：
 - 接收來自使用者或 **View** 的資料
 - 呼叫 **Model** 進行資料處理
 - 決定要回傳的結果或畫面
 - **Action** 方法執行完成後，通常會回傳 **ActionResult** 或其衍生類別，例如：**ViewResult**、**JsonResult**、**RedirectResult** 等。

製作與使用Action

- **Controller**下的方法為**Action**，可嘗試製作不同回傳型態的方法，並使用瀏覽器執行。

```
public string stringMethod()
{
    return "<h2>還是討厭下雨天</h2>";
}
```

```
public DateTime dtMethod()
{
    return DateTime.Now;
}
```

常見Content-Type (MIME Type) 對照表

Content-Type	說明	常見用途
text/plain	純文字	除錯、簡單回應
text/html	HTML 內容	回傳 HTML 片段
text/css	CSS 樣式	樣式表
text/javascript	JavaScript	腳本檔
application/json	JSON 格式	Web API 回傳
application/xml	XML 格式	舊系統整合
application/pdf	PDF 文件	文件下載
image/png	PNG 圖片	圖片回傳
image/jpeg	JPG 圖片	圖片回傳
application/octet-stream	二進位資料	檔案下載

Action的回傳種類-ContentResult

- Content方法：

- 回傳ContentResult類別的方法，可透過 Content-Type (MIME Type) 告訴瀏覽器「這段內容要怎麼解讀」。

```
public IActionResult ContentAction()  
{  
    return Content("<h2>蘋果讓你心靜</h2>", "text/html; charset=utf-8");  
}
```

- 設定ContentType可回傳不同的內容。

```
public IActionResult ContentAction()  
{  
    return Content("<h2>蘋果讓你心靜</h2>", "text/plain; charset=utf-8");  
}
```

Action的回傳種類-ViewResult

- **View方法：**
 - 回傳相對應的檢視頁面(View Page)。

```
public IActionResult Index()  
{  
    return View();  
}
```

- 回傳指定的檢視頁面。

```
public IActionResult redirectView()  
{  
    return View("Index");  
}
```

Action的回傳種類-FileResult

- **FilePathResult :**
 - 回傳實體路徑的檔案內容。
 - 也可使用**PhysicalFile()**。

```
public IActionResult ReturnFile()
{
    // 取得實際路徑
    //return File("~/images/mulan.png", "image/png");

    var path = Path.Combine(_env.WebRootPath, "images", "mulan.png");

    if (!System.IO.File.Exists(path))
        return NotFound($"File not found: {path}");

    return File(path, "application/octet-stream");
}
```

Action的回傳種類-JsonResult -1

- 將物件資料自動 序列化 (**Serialize**) 為 **JSON** 格式回傳給用戶端。
- 在 **ASP.NET Core** 中，**Controller** 回傳物件時通常會由框架自動輸出為 **JSON** (特別是 **ControllerBase / Web API** 專案) 。
- 可使用下列方式回傳 **JSON** 資料 (依情境選用) :
 - `return Ok(data);` (常用，回傳 200 + JSON)
 - `return new JsonResult(data);` (明確指定 JSON 回傳)
 - `return Json(data);` (**Minimal API** 常用)
- **ASP.NET Core** 允許 **GET** 回傳 **JSON** (舊版 **ASP.NET MVC** 預設禁止) 。

Action的回傳種類-JsonResult -2

```
public IActionResult ReturnJson()
{
    var myObject = new
    {
        Name = "Apple",
        Price = 150,
        Stock = 2000
    };

    return Ok(myObject);
    //return new JsonResult(myObject);
    //return Json(myObject);
}
```

Action的回傳種類-RedirectResult-1

- **Redirect()** :
 - 以「URL」作為目標進行重新導向 (**Redirect**) 。
 - 會回傳 302/301，讓瀏覽器重新發出新的請求到指定網址。
 - 適用：導向外部網站或已知的完整路徑。
 - **Redirect** 是導向 URL，不是直接導向 View。

```
public IActionResult redirectAction()
{
    return Redirect("~/About/Index");
}
```

Action的回傳種類-RedirectResult-2

- **RedirectToAction()** :
 - 以「Action 名稱 (可含 Controller) 」作為目標重新導向。
 - 由 MVC 依路由規則產生對應 URL，再進行 Redirect。
 - 適用：導向同站內其他 Action，維護性較好 (路由改了也較不怕壞)。
 - 可透過 Route Values 傳遞資料。
 - 可使用匿名物件將資料帶到目標 Action (Route Values)。

```
public IActionResult RedirectToAction()
{
    return RedirectToAction( "Index", "About");
}
```

```
public IActionResult RedirectToAction2()
{
    return RedirectToAction("ReturnMemberJson", "About", new {name = "金菜園", age=20});
}
0 個參考
public IActionResult ReturnMemberJson(string name, int age)
{
    var Member = new
    {
        Name = name,
        Age= age
    };
    return Ok(Member);
}
```

動作過濾與動作屬性 (Action / Attribute)

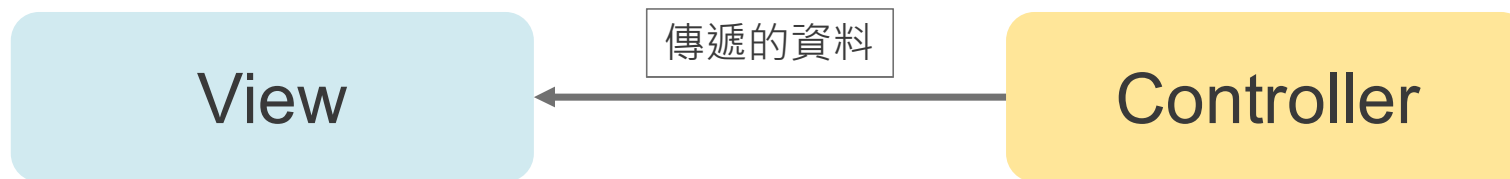
- 可透過 **Attribute** (標籤) 的方式，用來控制 **Action** 的行為與存取規則。
 - **[NonAction]**：標示某個方法 不被視為 **Action**該方法 不會被路由系統呼叫常用於 **Controller** 內的輔助方法 (**Helper Method**)。

```
[NonAction]
0 個參考
public void HelperMethod()
{
    // 這是一個輔助方法，不會被視為一個動作方法
}
```

- **[HttpGet]**、**[HttpPost]**、**[HttpDelete]**、**[HttpPut]**、**[HttpHead]**、**[HttpOptions]**、**[HttpPatch]**：用來限制 **Action** 可接受的 **HTTP** 通訊方式。

資料傳遞至View的方式

- 在 ASP.NET MVC 中，Controller 可透過以下三種方式將資料傳遞至檢視頁面（View）：
 - ViewData
 - TempData
 - ViewBag



ViewData

- 以 **Dictionary** (鍵值對) 方式，傳遞資料需進行型別轉換。
- 僅限於同一次 **RequestController**。

```
public IActionResult Index()
{
    ViewData["Message"] = "歡迎來到關於我們的頁面!";
    return View();
}
```

— View :

```
@{
    ViewBag.Title = "Index";
}

<h2>關於About</h2>
<h3>@ViewData["Message"]</h3>
```

TempData

- 可跨 Request 傳遞資料（例如 Redirect），內部使用 Session，資料在讀取後會被清除（一次性）。

```
public ActionResult PassData()
{
    TempData["CurrentTime"] = DateTime.Now.ToString();
    return View();
}
```

— View :

```
<h2>PassData</h2>

<h3>現在時間：@TempData["CurrentTime"]</h3>
```

ViewBag -1

- ViewData的語法糖（ Syntax Sugar ），使用 dynamic 屬性，寫法較簡潔，僅限於同一次 Request 。

```
public ActionResult PassData()
{
    ViewBag.CurrentTime = DateTime.Now;
    return View();
}
```

— View :

```
<h2>PassData</h2>

<h3>現在時間：@ViewBag.CurrentTime</h3>
```


ViewBag -2

- ViewData的Key與ViewBag的屬性可以共同使用。

```
public ActionResult PassData()
{
    ViewData["CurrentTime"] = DateTime.Now.ToString();
    return View();
}
```

— View :

```
<h2>PassData</h2>

<h3>現在時間(ViewBag) : @ViewBag.CurrentTime</h3>
<h3>現在時間(ViewData) : @ViewData["CurrentTime"]</h3>
```

應用程式名稱 首頁 關於 連絡人

PassData

現在時間(ViewBag) : 2021/1/8 下午 05:21:54

現在時間(ViewData) : 2021/1/8 下午 05:21:54

© 2021 - 我的 ASP.NET 應用程式

比較 ViewData 、 TempData 與 ViewBag-1

項目	ViewData	ViewBag	TempData
資料型態	Dictionary	Dynamic	Dictionary
型別安全	否	否	否
存活範圍	同一次 Request	同一次 Request	可跨 Request
常見用途	傳遞簡單資料	傳遞簡單資料	Redirect 訊息

比較 ViewData 、 TempData 與 ViewBag-2

- **Controller :**

```
public ActionResult CompareVDandTD()
{
    ViewData["PassbyVD"] = DateTime.Now.ToString();
    TempData["PassbyTD"] = DateTime.Now.ToString();
    ViewBag.PassbyVB = DateTime.Now;
    return RedirectToAction("PassDataView", "About");
}
0 個參考
public ActionResult PassDataView()
{
    return View();
}
```

比較 ViewData 、 TempData 與 ViewBag-3

- View :

```
PassDataView.cshtml  AboutController.cs  PassData.cshtml  _Layout.cshtml  Index.cshtml
1
2  @{
3      ViewBag.Title = "PassDataView";
4  }
5
6  <h2>PassDataView</h2>
7
8  <h3>現在時間( ViewData ) : @ViewData["PassbyVD"]</h3>
9  <h3>現在時間( TempData ) : @TempData["PassbyTD"]</h3>
10 <h3>現在時間( ViewBag ) : @ViewBag.PassbyVB</h3>
11
12
```

比較 ViewData 、 TempData 與 ViewBag-4

- 執行 Action- CompareVDandTD 的結果：

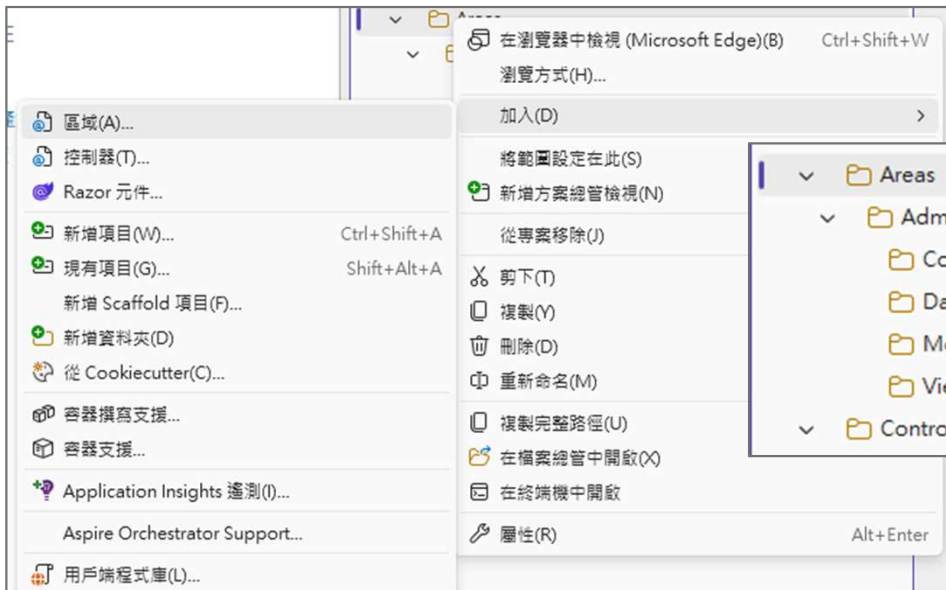


Area是什麼

- 適合：
 - 多模組系統 (**Sales / HR / Support**)
 - 同一專案要分組開發、避免 **Controller/Views** 互相踩到
- 不適合：
 - 只有 **1~2** 個頁面的小專案 (硬切反而麻煩)

建立 Areas 結構 -1

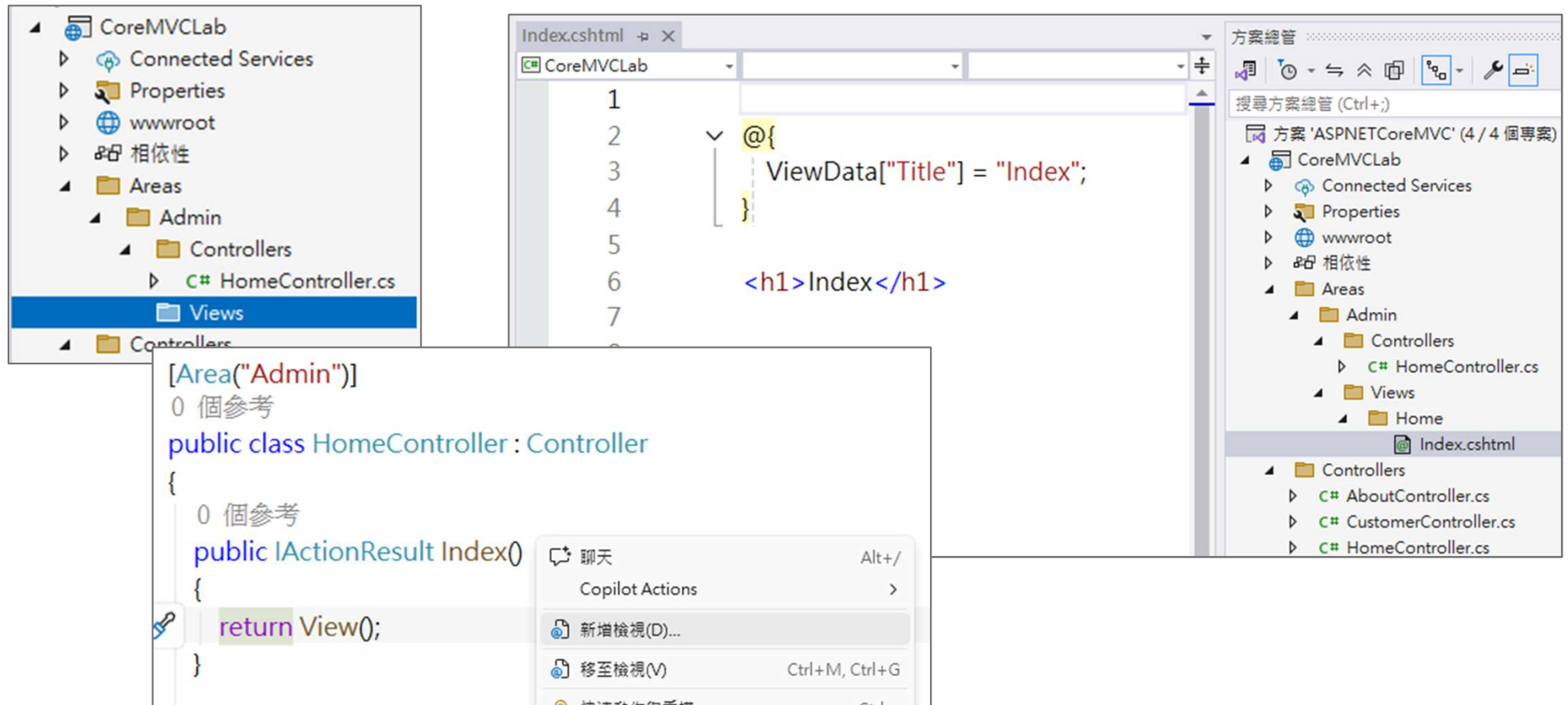
- 在專案目錄下建立Areas與模組資料夾。
- 建立Controller並設定Area名稱：
 - [Area("Admin")]：告訴 MVC 這個 Controller 屬於哪個 Area
 - [Route("Admin/...")]：讓網址變成 /Admin/Home/Index



```
[Area("Admin")]  
0 個參考  
public class HomeController : Controller  
{  
    0 個參考  
    public IActionResult Index()  
    {  
        return View();  
    }  
}
```

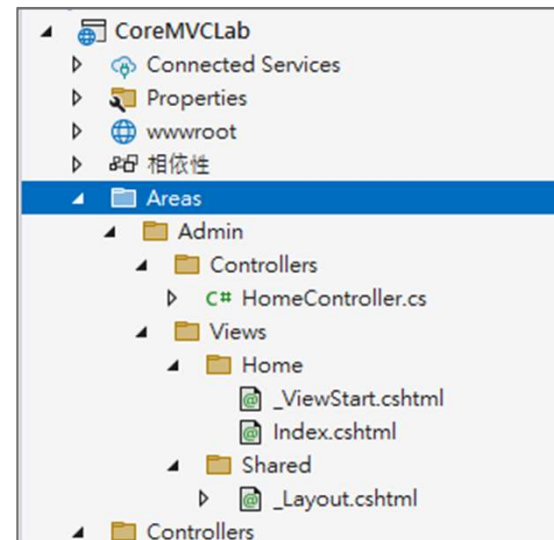
建立 Areas 結構 -2

- 在Areas資料夾下建立Views資料夾，並建立View頁面。



建立 Areas 結構 -3

- 可以想像Area就是將整個MVC架構縮小給各模組的資料夾架構。
- 可將Model、View的架構一起放上。



設定Area路由

- 在Program.cs設定Area路由。

```
app.MapControllerRoute(
    name: "areas",
    pattern: "{area:exists}/{controller=Home}/{action=Index}/{id?}");

app.MapControllerRoute(
    name: "sayHello",
    pattern: "Hello/{firstName}/{lastName?}",
    defaults: new { controller = "About", action = "SayHello" });

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```